

UART Communication Between STM32F103RB and Arduino Uno Using C

STM32F103RB Microcontroller Hardware Abstraction Layer (HAL)

Prepared By: Ahsan Basharat | Status: Intern | Submitted To: Sir Yasir

Objectives

- To understand the basic concept and working principle of UART (Universal Asynchronous Receiver/Transmitter) communication.
- To configure the USART peripheral of the STM32F103RB microcontroller using the STM32 HAL Library.
- To establish reliable serial communication between the STM32F103RB and an Arduino Uno.
- To transmit and receive data using HAL UART API functions.
- To gain hands-on experience in building, programming, debugging, and verifying UART communication using STM32CubeIDE and Arduino IDE.

Introduction

UART (Universal Asynchronous Receiver/Transmitter) is one of the most commonly used serial communication protocols in embedded systems. Unlike synchronous communication, UART does not require a clock signal between communicating devices. Instead, both devices communicate using pre-configured parameters such as baud rate, data bits, parity, and stop bits. UART is widely used for communication between microcontrollers, computers, sensors, GPS modules, Bluetooth modules, Wi-Fi modules, and other serial peripherals. The STM32 Hardware Abstraction Layer (HAL) simplifies UART peripheral configuration and data transmission by providing easy-to-use APIs, allowing developers to focus on application development rather than low-level register programming.

UART Communication Between STM32 and Arduino

In this experiment, UART communication is established between the STM32F103RB and an Arduino Uno. The STM32 transmits serial data through its USART1 peripheral (TX pin), while the Arduino receives the data on its RX pin and displays the received message using the Arduino Serial Monitor. Likewise, the Arduino can transmit data back to the STM32 through its TX pin. Both devices must share a common ground and operate with identical UART settings (115200 baud, 8 data bits, no parity, and 1 stop bit). This experiment demonstrates reliable asynchronous serial communication between two embedded systems using the STM32 HAL Library.

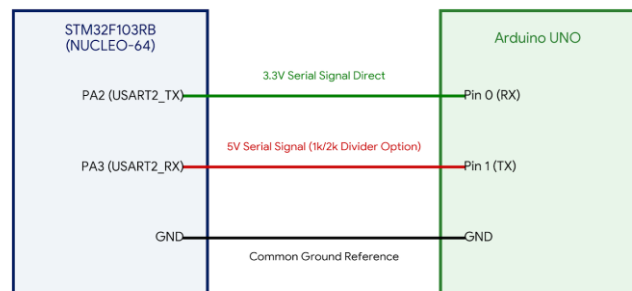
Required Components

#	Component / Tool	Quantity	Description / Specifications
1	STM32F103RB	1	ARM Cortex-M3 (32-bit)
2	USB Cable / ST-Link V2	1	Hardware programmer and debugger
3	Breadboard	1	Standard solderless prototyping breadboard
4	Light Emitting Diode (LEDs)	2	5mm Red/Green LED indicator
5	Jumper Wires	Set	Male-to-Male and Male-to-Female wires

6	STM32CubeIDE/MX	Software	Integrated Development Environment for C/C++
7	Arduino Uno	1	ATmega328P Development Board
8	Arduino IDE	Software	Upload Arduino Sketch and Serial Monitor

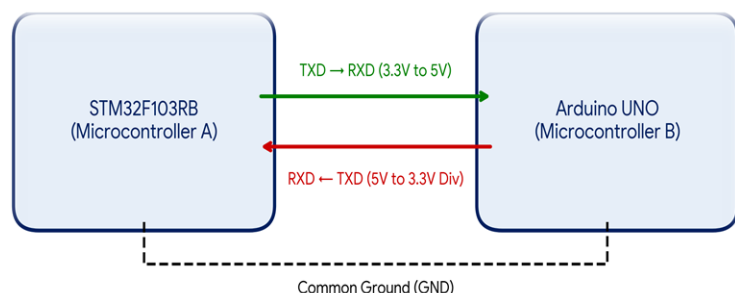
Circuit Description

- **UART TX Connection:** STM32 USART1 TX pin (PA9) is connected to the Arduino Uno RX (Digital Pin 0).
- **UART RX Connection:** STM32 USART1 RX pin (PA10) is connected to the Arduino Uno TX (Digital Pin 1).
- **Ground Connection:** The GND pin of the STM32 board is connected to the GND pin of the Arduino Uno to establish a common reference voltage.
- **UART Configuration:** Both devices are configured with identical communication parameters:
 - Baud Rate: 115200 bps
 - Data Bits: 8
 - Stop Bits: 1
 - Parity: None
 - Flow Control: None



UART Communication Between STM32F103RB and Arduino UNO

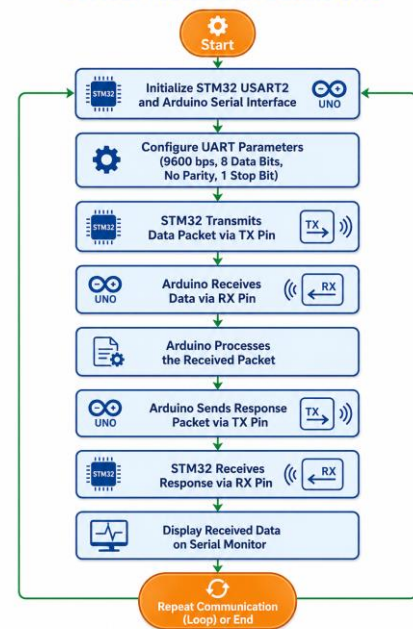
The diagram illustrates the UART serial communication interface between an STM32F103RB microcontroller and an Arduino UNO. The STM32 transmits data at 3.3 V, which is directly accepted by the Arduino UNO's 5 V logic input. However, the Arduino's 5 V transmit signal must be reduced to 3.3 V using a voltage divider before reaching the STM32 receiver to prevent damage. Both microcontrollers share a common ground (GND), ensuring a stable reference voltage and reliable bidirectional communication. This configuration enables safe and efficient data exchange between the two devices.



UART Flow Graph

The flowchart illustrates the complete UART communication sequence between the STM32F103RB microcontroller and the Arduino UNO. It begins with UART initialization and parameter configuration, followed by bidirectional data transmission and reception through the TX and RX pins. After processing the received data, the Arduino sends a response back to the STM32, and the communication continues in a loop until the process is terminated.

UART Communication Flow Between STM32F103RB and Arduino UNO



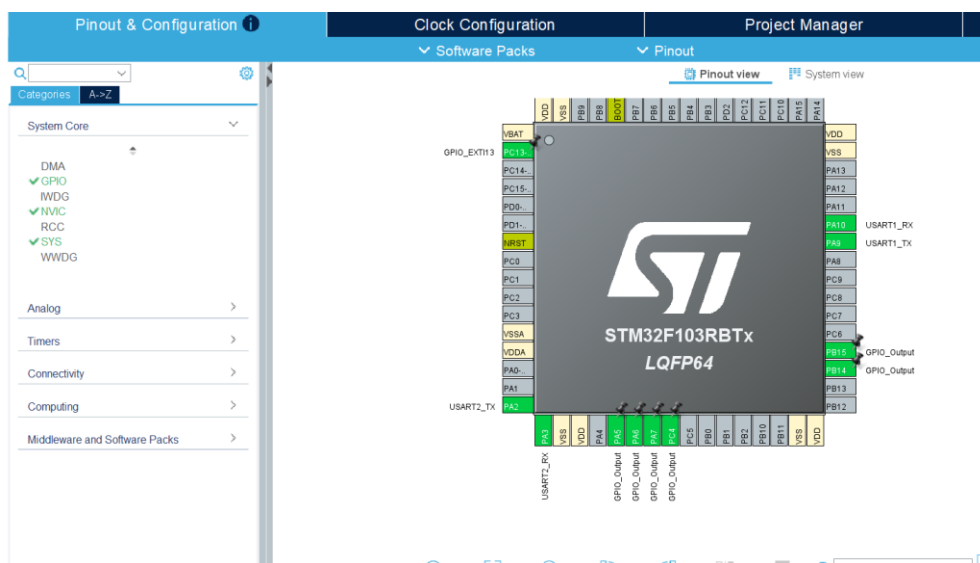
Procedure

1. Launch **STM32CubeIDE** and create a new project by selecting the **STM32F103RB** microcontroller.
2. Open the .ioc configuration file and enable **USART1** in **Asynchronous Mode**.
3. Configure the UART peripheral with the following settings:
 - **Baud Rate:** 115200
 - **Word Length:** 8 Bits
 - **Stop Bits:** 1
 - **Parity:** None
 - **Mode:** TX/RX
4. Save the IOC file and allow STM32CubeIDE to generate the initialization code.
5. Write the UART communication program using HAL functions such as **HAL_UART_Transmit()** and **HAL_UART_Receive()**.
6. Open **Arduino IDE** and write a sketch to receive serial data using **Serial.begin(115200)** and **Serial.read()**.
7. Upload the Arduino program to the Arduino Uno.
8. Connect the hardware:
 - STM32 **PA9 (TX)** → Arduino **RX (Pin 0)**
 - STM32 **PA10 (RX)** → Arduino **TX (Pin 1)**
 - STM32 **GND** → Arduino **GND**
9. Program the STM32F103RB using the **ST-Link V2** debugger.
10. Open the **Arduino Serial Monitor** and set the baud rate to **115200 bps**.
11. Observe the transmitted and received messages to verify successful UART communication between the STM32 and Arduino.

HAL Functions Used

Function Name	Purpose
UART_Init()	Initializes the UART peripheral with the configured communication parameters and prepares it for data transmission and reception.
UART_SendChar()	Transmits a single character over the UART interface using the STM32 HAL blocking transmit function.
UART_SendString()	Transmits a null-terminated string over the UART interface by sending each character sequentially.
UART_ReadChar()	Receives a single character from the UART interface using the STM32 HAL blocking receive function.
HAL_UART_Transmit()	Transmits one or more bytes of data over UART in blocking mode
HAL_UART_Receive()	Receives one or more bytes of data over UART in blocking mode

Code:



```

typedef struct
{
    USART_TypeDef *Instance; /* UART peripheral instance */
    UART_HandleTypeDef Handle; /* HAL UART handle */
} UART_HandleTypeDef;

HAL_StatusTypeDef UART_SendString(UART_HandleTypeDef * const uart, const char *);

/**
 * @brief Receive one character.
 *
 * @param uart Pointer to UART object.
 * @param ch Pointer to received character.
 * @return HAL status.
 */
HAL_StatusTypeDef UART_ReadChar(UART_HandleTypeDef * const uart, char *ch);

/*
 * Function Prototypes
 */

/**
 * @brief Initialize UART peripheral.
 *
 * @param uart Pointer to UART object.
 */
void UART_Init(UART_HandleTypeDef * const uart);

/**
 * @brief Transmit one character.
 *
 * @param uart Pointer to UART object.
 * @param ch Character to transmit.
 * @return HAL status.
 */
HAL_StatusTypeDef UART_SendChar(UART_HandleTypeDef * const uart, char ch);

/**
 * @brief Transmit a null-terminated string.
 *
 * @param uart Pointer to UART object.
 * @param str Pointer to string.
 * @return HAL status.
 */
HAL_StatusTypeDef UART_SendString(UART_HandleTypeDef * const uart, const char *str);

/**
 * @brief Receive one character.
 *
 * @param uart Pointer to UART object.
 * @param ch Pointer to received character.
 * @return HAL status.
 */
HAL_StatusTypeDef UART_ReadChar(UART_HandleTypeDef * const uart, char *ch);

```

```

/* *****
 * @brief Initialize UART peripheral.
 *
 * @param uart Pointer to UART object.
 *
 * @note Configures USART1 with:
 * BaudRate : 115200
 * WordLength : 8 Bits
 * StopBits : 1
 * Parity : None
 * Mode : TX/RX
 * *****
void UART_Init(UART_HAL * const uart)
{
    if (uart == NULL)
    {
        return;
    }

    /* Enable peripheral clock */
    if (uart->Instance == USART1)
    {
        __HAL_RCC_USART1_CLK_ENABLE();
    }

    uart->Handle.Instance = uart->Instance;

    /* UART Configuration */
    uart->Handle.Init.BaudRate = 115200U;
    uart->Handle.Init.WordLength = UART_WORDLENGTH_8B;
    uart->Handle.Init.StopBits = UART_STOPBITS_1;
    uart->Handle.Init.Parity = UART_PARITY_NONE;
    uart->Handle.Init.Mode = UART_MODE_TX_RX;
    uart->Handle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    uart->Handle.Init.OverSampling = UART_OVERSAMPLING_16;

    (void)HAL_UART_Init(&uart->Handle);
}

```

```

/* *****
 * @brief Transmit one character.
 * *****
HAL_StatusTypeDef UART_SendChar(UART_HAL * const uart,
                                char ch)
{
    return HAL_UART_Transmit(&uart->Handle, (uint8_t *)&ch, 1U, UART_TIMEOUT);
}

/* *****
 * @brief Transmit string.
 * *****
HAL_StatusTypeDef UART_SendString(UART_HAL * const uart,
                                  const char *str)
{
    return HAL_UART_Transmit(&uart->Handle, (uint8_t *)str, (uint16_t)strlen(str), UART_TIMEOUT);
}

/* *****
 * @brief Receive one character.
 * *****
HAL_StatusTypeDef UART_ReadChar(UART_HAL * const uart,
                                char *ch)
{
    return HAL_UART_Receive(&uart->Handle, (uint8_t *)ch, 1U, HAL_MAX_DELAY);
}

```

```

1  #include "main.h"
2  #include "usart.h"
3  #include "gpio.h"
4  #include "GPIO_HAL.h"
5  // #include "interr_fun.h"
6  #include "UART_HAL.h"
7  #include <stdio.h>
8  void SystemClock_Config(void);
9  UART_HAL DebugUART =
10 {
11     USART1
12 };
13
14 char Rx;
15 LEDS LED1 =
16 {
17     GPIOA,
18     GPIO_PIN_5
19 };
20
21 LEDS LED2 =
22 {
23     GPIOA,
24     GPIO_PIN_6
25 };
26
27 LEDS LED3 =
28 {
29     GPIOA,
30     GPIO_PIN_7
31 };
32
33

```

```

1  // *****
2  {
3  }
4  SystemClock_Config();
5  MX_USART1_UART_Init();
6  MX_USART2_UART_Init();
7  }
8  LED_Init(&LED1);
9  LED_Init(&LED2);
10 LED_Init(&LED3);
11 LED_Init(&LED4);
12 LED_Init(&LED5);
13 LED_Init(&LED6);
14 }
15 //LED_Init(&Button);
16
17 UART_Init(&DebugUART);
18 UART_SendString(&DebugUART,
19                 "UART Driver Ready\r\n");
20 }
21 while (1)
22 {
23     // LED_TogglePin(&LED1);
24     // LED_TogglePin(&LED2);
25     // LED_TogglePin(&LED3);
26     // LED_TogglePin(&LED4);
27     // LED_TogglePin(&LED5);
28     // LED_TogglePin(&LED6);
29     // HAL_Delay(1000);
30 }
31 // if (UART_ReadChar(&DebugUART, &Rx) == HAL_OK)
32 // {
33 //     UART_SendChar(&DebugUART, Rx);
34 // }
35 // if (UART_SendString(&DebugUART, "UART Driver Ready\r\n") == HAL_OK)
36 // {
37 //     LED_TogglePin(&LED4);
38 //     HAL_Delay(1000);
39 // }
40 }

```

Aurdino Uno code

```
#include <SoftwareSerial.h>

#define LED_PIN 13

// RX, TX
SoftwareSerial STM32(2, 3);

void setup()
{
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.begin(115200);    // PC Serial Mon:
    STM32.begin(115200);    // UART to STM32

    Serial.println("Arduino UART Ready");
}

void loop()
{
    // Receive data from STM32
    if (STM32.available())
    {
        String data = STM32.readStringUntil('\n');

        Serial.print("Received: ");
        Serial.println(data);

        // Blink LED when data is received
        digitalWrite(LED_PIN, HIGH);
        delay(100);
        digitalWrite(LED_PIN, LOW);
    }

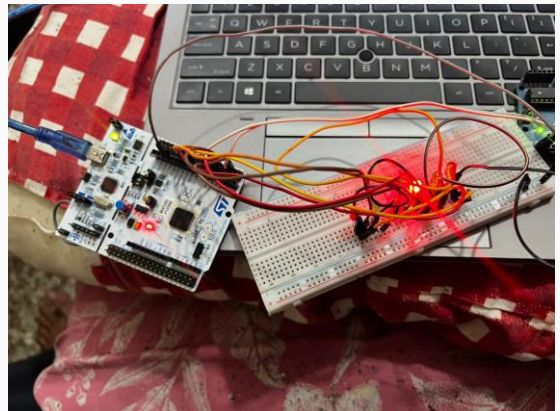
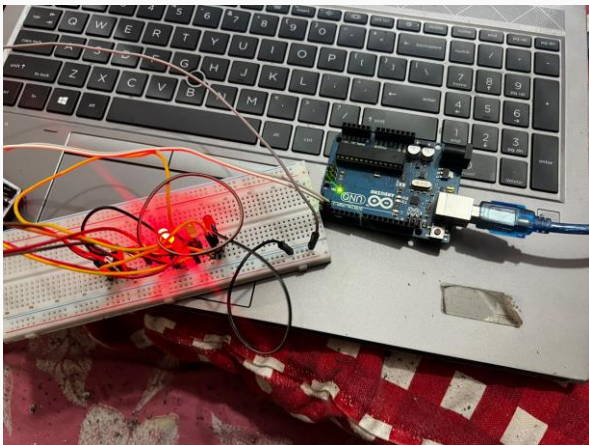
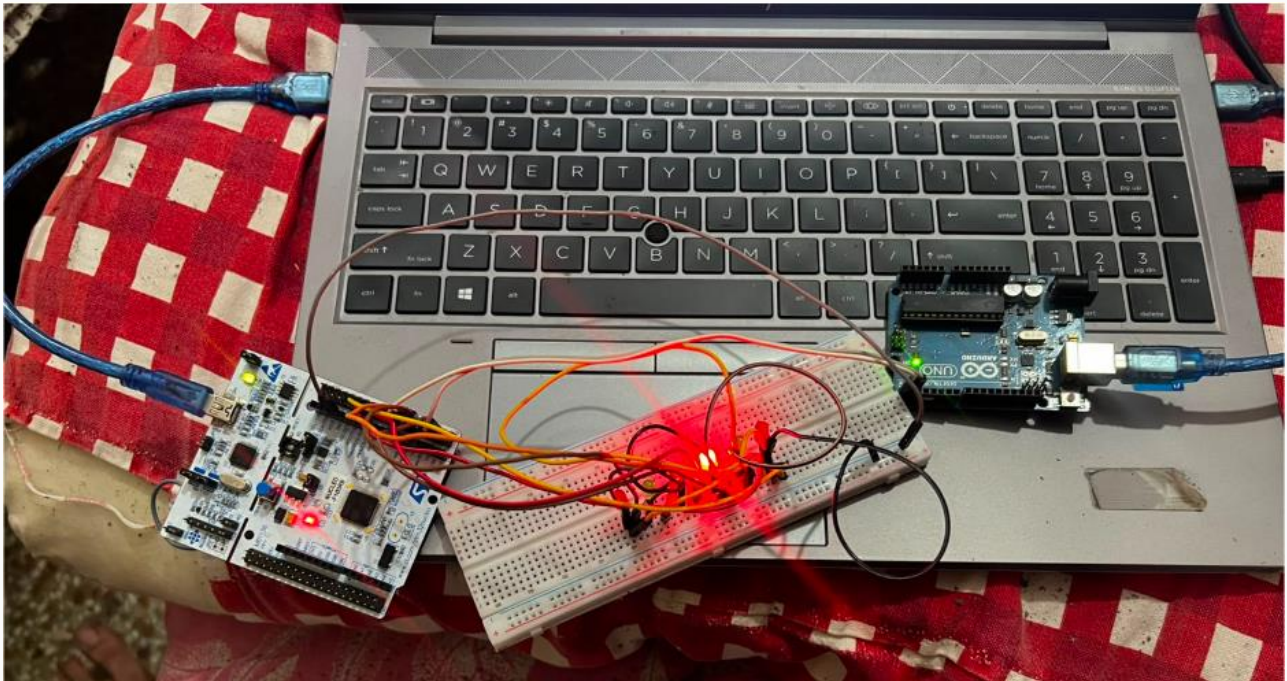
    // Send data to STM32
    if (Serial.available())
    {
        String data = Serial.readStringUntil('\n');

        STM32.println(data);

        Serial.print("Sent: ");
        Serial.println(data);

        // Blink LED when data is sent
        digitalWrite(LED_PIN, HIGH);
        delay(100);
        digitalWrite(LED_PIN, LOW);
    }
}
```

Program Output



```
03:22:55.651 -> Received from STM32: UART Driver Ready
03:22:55.651 -> Received from STM32: UART Driver Ready
03:22:56.630 -> Received from STM32: UART Driver Ready
03:22:57.630 -> Received from STM32: UART Driver Ready
03:22:58.654 -> Received from STM32: UART Driver Ready
03:22:59.670 -> Received from STM32: UART Driver Ready
03:23:00.657 -> Received from STM32: UART Driver Ready
03:23:01.637 -> Received from STM32: UART Driver Ready
03:23:02.657 -> Received from STM32: UART Driver Ready
03:23:03.641 -> Received from STM32: UART Driver Ready
03:23:04.669 -> Received from STM32: UART Driver Ready
03:23:05.684 -> Received from STM32: UART Driver Ready
03:23:06.669 -> Received from STM32: UART Driver Ready
03:23:07.649 -> Received from STM32: UART Driver Ready
03:23:08.681 -> Received from STM32: UART Driver Ready
03:23:09.684 -> Received from STM32: UART Driver Ready
03:23:10.684 -> Received from STM32: UART Driver Ready
03:23:11.679 -> Received from STM32: UART Driver Ready
03:23:12.685 -> Received from STM32: UART Driver Ready
03:23:13.687 -> Received from STM32: UART Driver Ready
03:23:14.665 -> Received from STM32: UART Driver Ready
03:23:15.679 -> Received from STM32: UART Driver Ready
03:23:16.700 -> Received from STM32: UART Driver Ready
03:23:17.668 -> Received from STM32: UART Driver Ready
```

Result

- The UART peripheral on both the STM32F103RB and Arduino Uno was successfully initialized.
- Both devices were configured with the same baud rate (115200 bps), ensuring reliable serial communication.
- The STM32 successfully transmitted the message "UART Driver Ready" to the Arduino using the `UART_SendString()` function.
- The Arduino correctly received the transmitted data and displayed it on the Serial Monitor.
- The STM32 LED4 blinked after each successful transmission, confirming that data was sent successfully.
- The Arduino built-in LED (Pin 13) blinked whenever data was received from the STM32, indicating successful reception.
- The Arduino also transmitted user-entered data from the Serial Monitor back to the STM32 through UART.
- The transmitted and received messages matched correctly, confirming error-free bidirectional communication.
- The experiment verified successful full-duplex UART communication between the STM32F103RB and Arduino Uno using the STM32 HAL Library.

Conclusion

The UART communication experiment was successfully implemented between the STM32F103RB and Arduino Uno using the STM32 HAL Library. Serial data was transmitted from the STM32 and correctly received by the Arduino, while data sent from the Arduino was also successfully received by the STM32. LED indicators on both devices confirmed successful transmission and reception, and the Arduino Serial Monitor verified the exchanged messages. This experiment demonstrated reliable asynchronous serial communication and provided practical experience in configuring and using UART peripherals in embedded systems.

Github Repository:

<https://github.com/Ahsan-web-hue/UART-Between-STN32NF103RB-and-Aurdino-UNO.git>